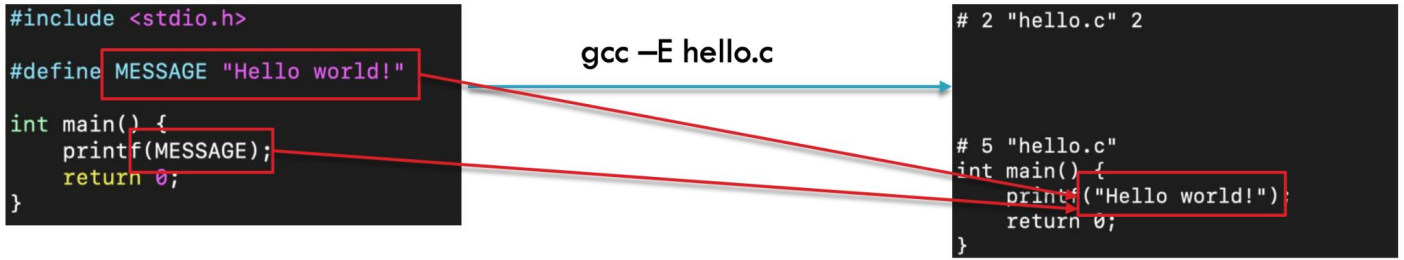
Software Security

University of Insubria

From code to programs

Compiling a C program is a multi-stage process

- **preprocessing**
  - handles preprocessor commands (like #include, #define)



From code to programs

Compiling a C program is a multi-stage process

- **compilation**
  - converts code to assembly language

From code to programs

Compiling a C program is a multi-stage process

- **assembly**
  - translates assembly into machine (object) code

From code to programs

Compiling a C program is a multi-stage process

- **linking**
  - combines object code into a final executable (links to the used library)
  - ***static***: all code is included in the binary
  - ***dynamic***: uses shared libraries loaded at runtime

ELF - Executable and Linkable Format

ELF (Executable and Linkable Format)

- standard file format used for C programs

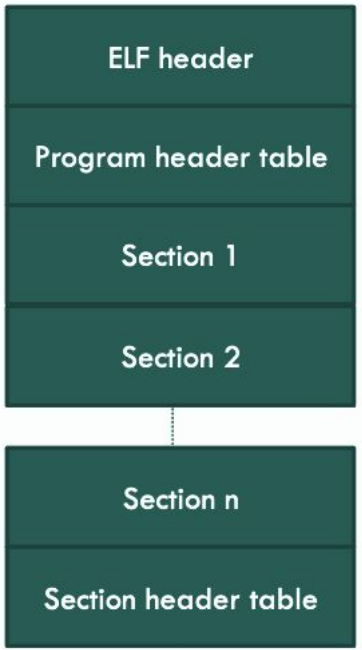
Types of ELF Files

- **Relocatable Files**
  - contain code/data that can be linked with other files
- **Executable files**
  - ready-to-run programs
- **Shared object files**
  - can be linked with other files
  - used by the system at runtime (dynamic linking)

ELF - Executable and Linkable Format

Any ELF file is structured as

- **ELF Header**
  - basic info about the file (type, architecture, etc.)
- **Program Header Table**
  - tells the system how to create a running program (process image)
- (sequence of) **Sections**
  - contain code, data, and other info needed for linking
- **Section Header Table**
  - describes each section in the file



ELF - Executable and Linkable Format

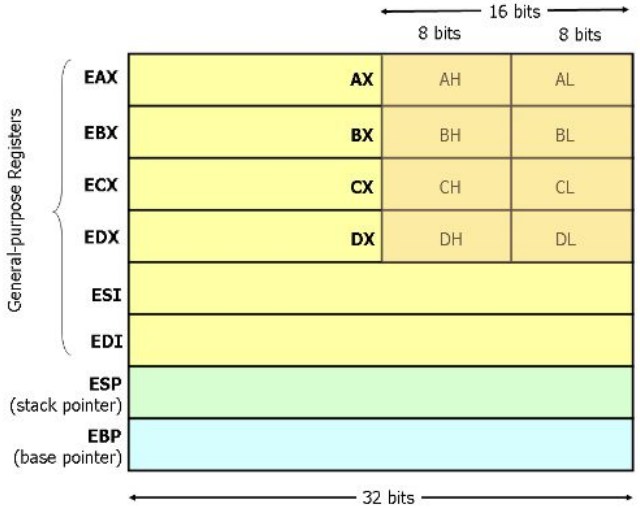
Common ELF Sections

- **.text**: contains the program's executable code
- **.bss**: uninitialized data (gets memory at runtime, but no value yet)
- **.data/.data1**: initialized data used by the program
- **.rodata/.rodata1**: read-only data (e.g., constants, strings)
- **.symtab**: contains the symbol table (info about functions and variables)
- **.dynamic**: contains info used for dynamic linking

X86-32 Registers

x86-32 processors have eight 32-bit general purpose registers

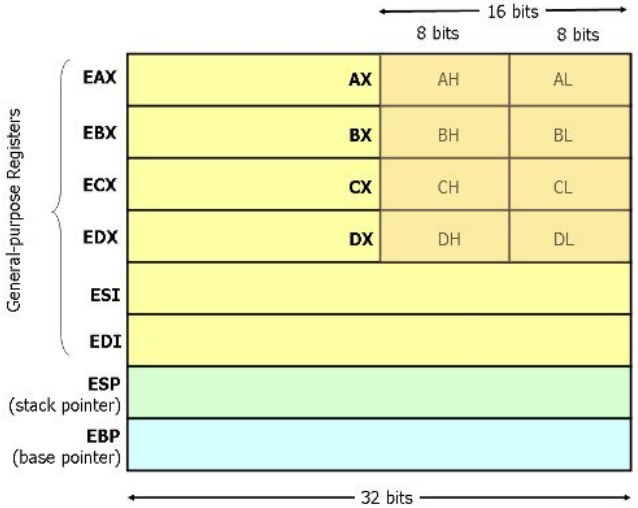
- **EAX**
  - accumulator (used for arithmetic operations and return values from functions)
- **EBX**
  - Base register (used in addressing)
- **ECX**
  - Counter (often used in loop indices)
- **EDX**:
  - Data register (used for I/O, and arithmetic operations)
- **ESI**
  - Source index (used in string operations like movs)



X86-32 Registers

x86-32 processors have eight 32-bit general purpose registers

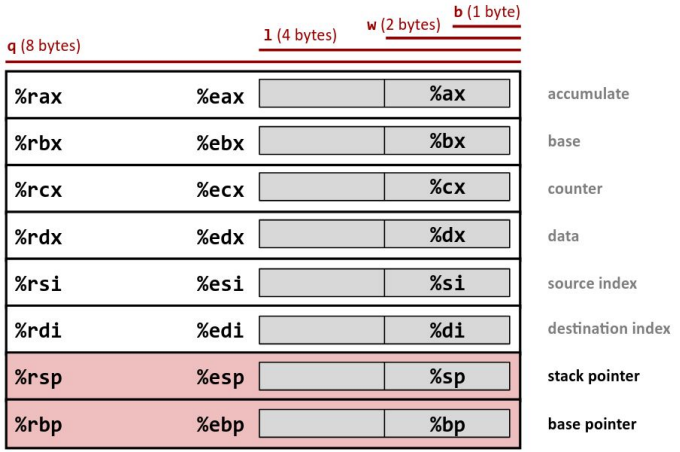
- **EDI**
  - Destination index (used in string operations like movs)
- **ESP**
  - Stack pointer (points to the top of the stack)
- **EBP**
  - Base pointer (used to reference function parameters and local variables)
- **EIP**
  - Instruction pointer (holds the address of the next instruction to be executed)



X86-64 Registers

x86-64 processors have sixteen 64-bit general purpose registers

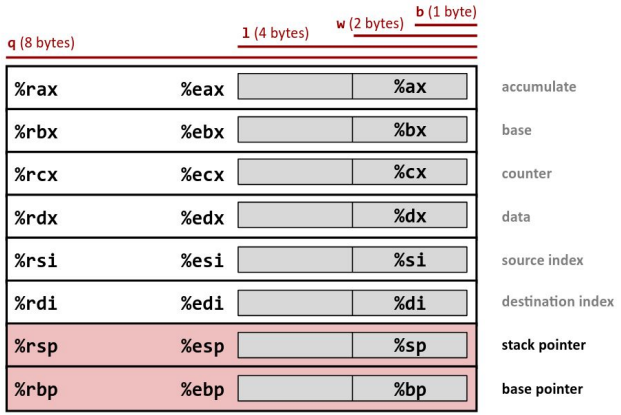
- **RAX**
  - accumulator (used for arithmetic operations and return values from functions)
- **RBX**
  - Base register (used in addressing)
- **RCX**
  - Counter (often used in loop indices)
- **RDX**:
  - Data register (used for I/O, and arithmetic operations)
- **RSI**
  - Source index (used in string operations like movs)



X86-64 Registers

x86-64 processors have sixteen 64-bit general purpose registers

- **RDI**
  - Destination index (used in string operations like movs)
- **RSP**
  - Stack pointer (points to the top of the stack)
- **RBP**
  - Base pointer (used to reference function parameters and local variables)
- **R8–R15**:
  - Additional registers (typically used for passing arguments)
- **RIP**
  - Instruction pointer (holds the address of the next instruction to be executed)



x86 Instructions

Data Movement

- **mov op1, op2** -> Copy data from op2 to op1
- **push op** -> Put op on top of the stack
- **pop op** -> Remove top item from stack into op
- **lea reg, addr** -> Load address into register (not the value)

Arithmetic and Logic

- **add op1, op2** -> op1 = op1 + op2
- **sub op1, op2** -> op1 = op1 - op2
- **and / or / xor op1, op2** -> Bitwise operations on op1 and op2

x86 Instructions

Control Flow

- **jmp op** -> Unconditional jump to the instruction at op
- **cmp op1, op2** -> Compares op1 and op2, updates status flags (no actual result stored)
- **j<condition> op** -> Conditional jump based on comparison result
  - je (jump if equal)
  - jne (jump if not equal)
  - jg (jump if greater)
  - jl (jump if less)

Memory management

High-level languages (like Python, Java) hide memory details from programmers

- Programmers often ignore where data is allocated (compiler makes decisions)

In C, programmers have more control over memory

- Dynamically allocate and free memory
  - memory can be dynamically allocated and deallocated (free)
- Access memory addresses
  - memory address of variables can be obtained (if x is a variable, &x denotes the pointer to x)

Memory allocation

```c
#include <stdio.h>

int main() {
    int i;
    char c;
    short s;
    long l;

    printf("i is allocated at %p\n", &i);
    printf("c is allocated at %p\n", &c);
    printf("s is allocated at %p\n", &s);
    printf("l is allocated at %p\n", &l);
}
```

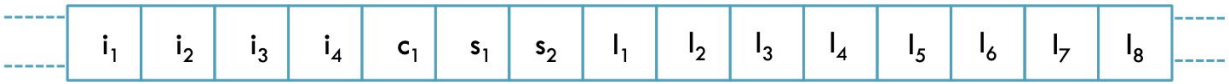
C program:

- **int** needs 4 bytes
- **char** needs 1 byte
- **short** needs 2 bytes
- **long** needs 8 bytes

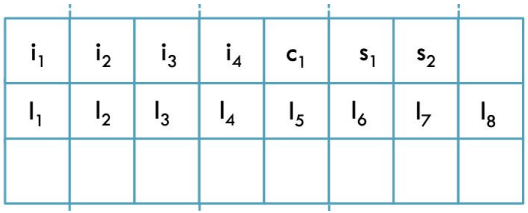
```
CC> gcc -o alignment alignment.c
CC> ./alignment
i is allocated at 0x7ffee117e7cc
c is allocated at 0x7ffee117e7cb
s is allocated at 0x7ffee117e7c8
l is allocated at 0x7ffee117e7c0
CC>
```

Memory allocation

Memory is usually represented as a sequence of bytes



In a n·8 architecture, bytes are arranged in groups of n

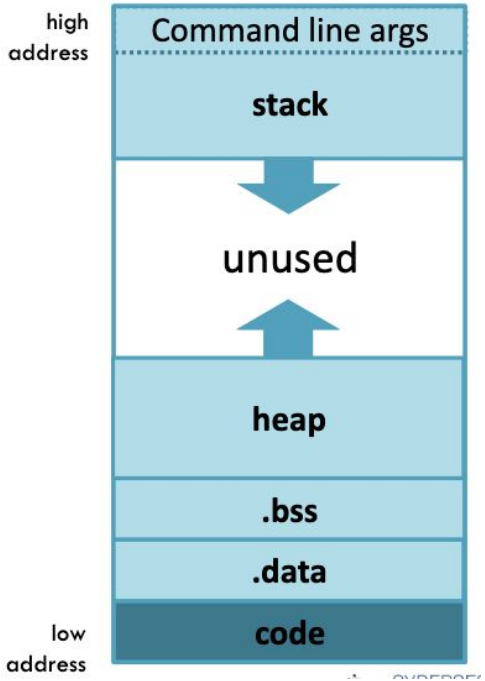


Compilers may introduce padding or change the order of data in memory (optional optimization)

Memory segments

Memory is allocated for each process (a running program) to store data and code. Different segments

- **Stack**
  - Stores local variables (automatically managed)
- **Heap**
  - Used for dynamically allocated memory
- **Data Segment**
  - .bss -> Uninitialized global variables
  - .data -> Initialized global variables
- **Code segment**
  - **Stores the program's executable code**

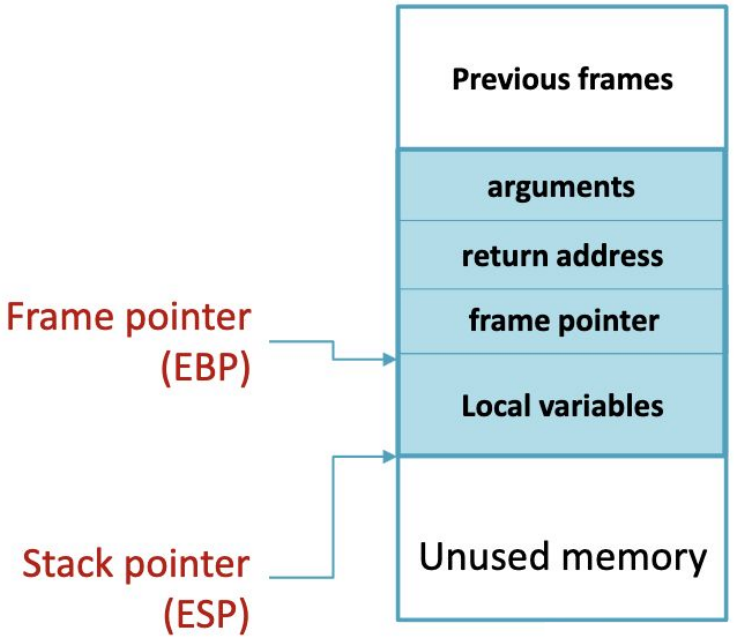


The stack

The stack consists of a sequence of stack frames, each for each function call (allocated on call, de-allocated on return)

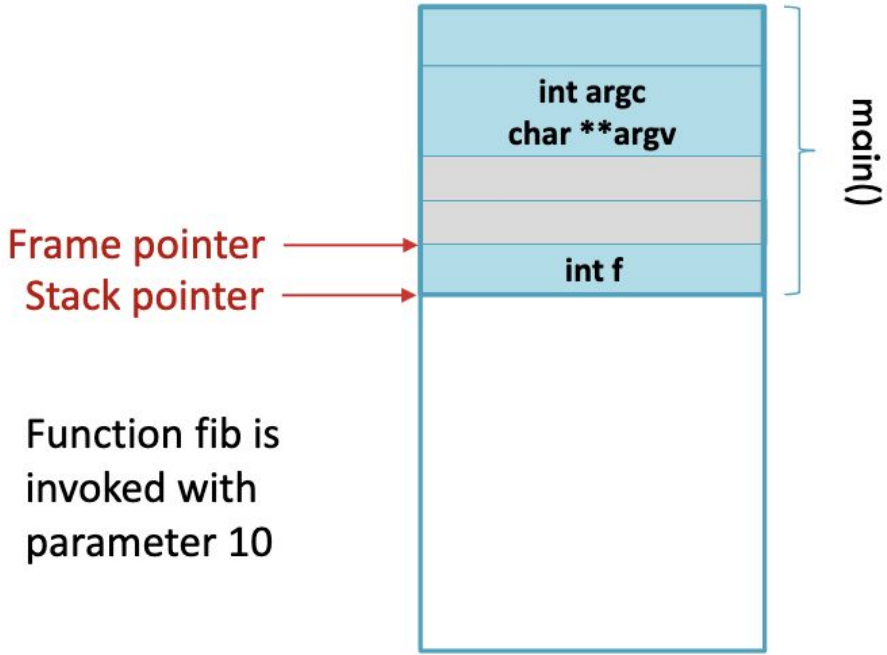
Pointers

- **stack pointer (RSP)**
  - Points to the last element on the stack
- **frame pointer (RBP)**
  - Points to the start of the current function's local variables



Stack frame

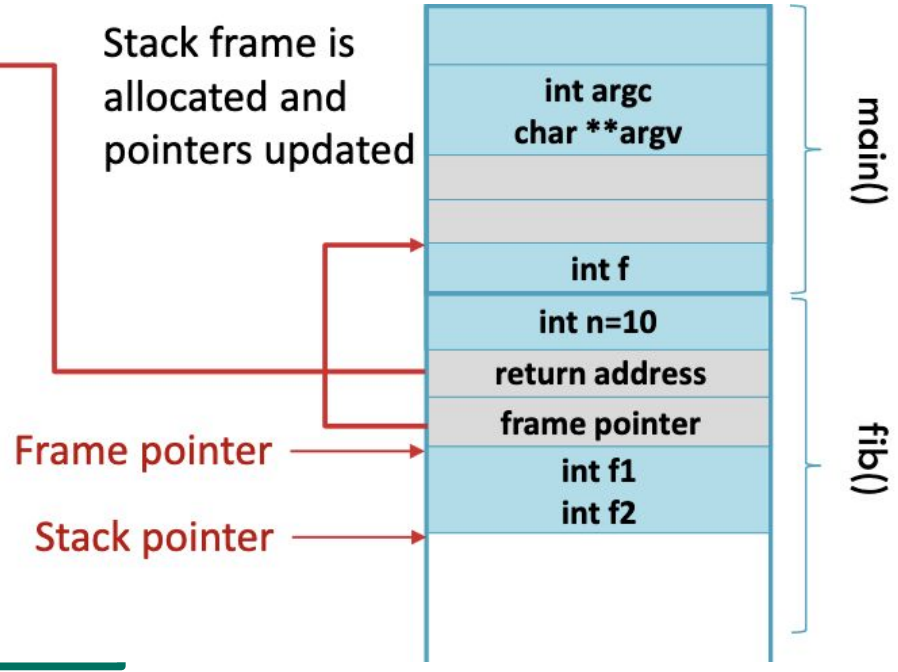
```
int main(int argc, char **argv) {
  int f = fib( n: 10);
  printf("FIB(10)=%d\n",f);
}

int fib(int n) {
  int f1;
  int f2;
  if (n<=2) {
      return 1;
  } else {
      f1 = fib( n: n-1);
      f2 = fib( n: n-2);
      return f1+f2;
  }
}
```



Function fib is invoked with parameter 10

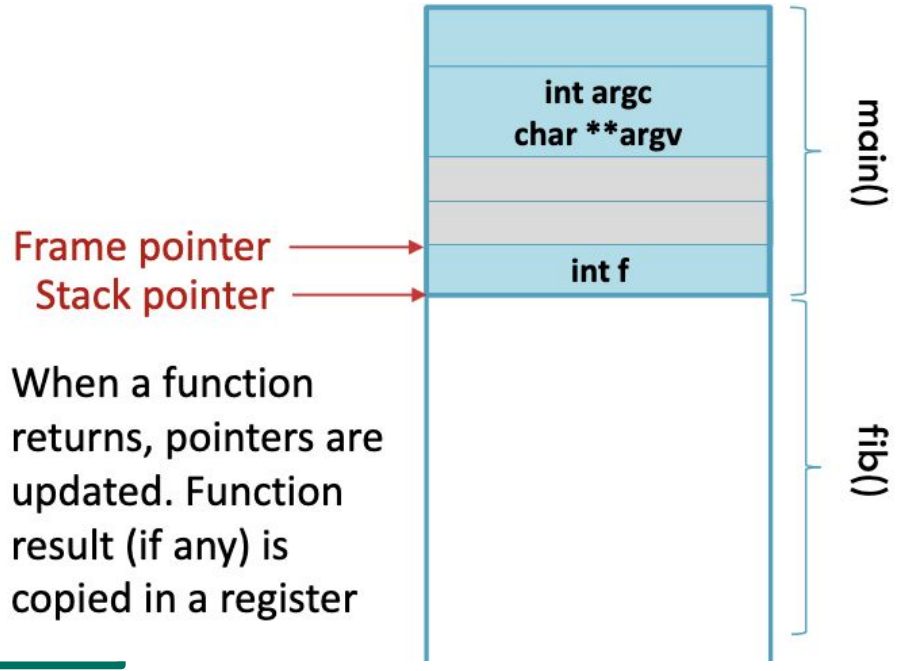
Stack frame

```
int main(int argc, char **argv) {
  int f = fib( n: 10);
  printf("FIB(10)=%d\n",f);
}

int fib(int n) {
  int f1;
  int f2;
  if (n<=2) {
      return 1;
  } else {
      f1 = fib( n: n-1);
      f2 = fib( n: n-2);
      return f1+f2;
  }
}
```



Stack frame

```
int main(int argc, char **argv) {
  int f = fib( n: 10);
  printf("FIB(10)=%d\n",f);
}

int fib(int n) {
  int f1;
  int f2;
  if (n<=2) {
      return 1;
  } else {
      f1 = fib( n: n-1);
      f2 = fib( n: n-2);
      return f1+f2;
  }
}
```



When a function returns, pointers are updated. Function result (if any) is copied in a register

Memory management functions

Basic C functions for memory management

- **malloc(int n)**
  - Allocates n bytes of memory and returns a pointer to it
- **free(void* ptr)**
  - Deallocates memory pointed to by ptr

Debugging

Debugging in Software

- Finding and fixing bugs (problems that prevent correct operation)

Compilers can generate extra data for debuggers to provide more info (e.g., use -g with GCC)

- Debugging Information is stored in ELF file sections
  - .debug: contains data for symbolic debugging
  - .line: contains line number info for mapping code to source lines